

WEEK 3 – PYTHON & DATA WRANGLING

FINAL SUBMISSION

Project: Cleaning and analyzing a messy dataset using Python, Pandas, Matplotlib and Seaborn

Item	Completed result
Source	data.csv
Initial dataset	32 rows × 5 columns
Final dataset	31 rows × 7 columns
Missing values after cleaning	0
Duplicate rows after cleaning	0
Python libraries	Pandas, NumPy, Matplotlib, Seaborn
Notebook used	Week 3 Python Data Wrangling.ipynb

This submission is based directly on the uploaded and executed Week 3 Python Data Wrangling notebook. The report presents the completed data inspection, cleaning, missing-value treatment, duplicate handling, filtering, derived columns, summary statistics and visualizations.

1. Objective and Dataset Inspection

The Week 3 task focuses on Python and data wrangling. The completed notebook reads the supplied CSV into Pandas, inspects the structure and data types, identifies data-quality issues, cleans the dataset, filters records, creates new columns and produces basic visualizations.

Initial dataset

```
df = pd.read_csv("data.csv")
df.head()
```

Inspection	Executed result
Shape	(32, 5)
Columns	Duration, Date, Pulse, Maxpulse, Calories
Date type	object
Calories type	float64
Numeric columns	Duration, Pulse, Maxpulse

Initial data-quality findings

Issue	Finding
Missing Date	1 value
Missing Calories	2 values
Duplicate rows	1 duplicate
Mixed Date representations	Present
Extreme Duration	450 minutes

2. Handling Missing Values

Missing-value check

```
df.isnull().sum()
```

The initial check identified one missing Date value and two missing Calories values. The other three columns had no missing values.

Date cleaning

```
df["Date"] = pd.to_datetime(  
df["Date"].astype(str).str.strip(""),  
format="mixed",  
errors="coerce"  
)
```

The Date column was converted to datetime. The executed notebook detected `np.int64(1)` missing date value. Because the surrounding records follow a daily sequence, the missing date was restored as 22 December 2020.

```
missing_date_index = df["Date"].isna()  
  
df.loc[missing_date_index, "Date"] = pd.Timestamp("2020-12-22")
```

Missing Date values after the correction: `np.int64(0)`.

Calories cleaning

```
calories_median = df["Calories"].median()  
calories_median
```

The executed notebook calculated a median Calories value of **291.2** and used it to replace the two missing Calories values.

```
df["Calories"] = df["Calories"].fillna(calories_median)
```

Missing-value validation

```
df.isnull().sum()
```

After cleaning, every original column has zero missing values.

3. Duplicate and Outlier Check

Duplicate handling

```
df.duplicated().sum()
```

Duplicate rows before removal: **np.int64(1)**.

```
df = df.drop_duplicates()
```

Duplicate rows after removal: **np.int64(0)**.

The duplicate record was removed to prevent the same observation from being counted twice.

Outlier inspection

```
df["Duration"].describe()
```

The Duration statistics show a maximum of 450 minutes, which is substantially larger than the median and most observations.

```
df[df["Duration"] > 120]
```

The 450-minute record was identified and retained rather than silently deleted. The Week 3 task requires data cleaning, missing-value handling, filtering and new-column creation, but does not require arbitrary outlier deletion. Documenting the observation preserves the original information while making the quality issue visible.

4. Creating New Columns

Calories per minute

```
df["Calories_per_Minute"] = df["Calories"] / df["Duration"]
```

Calories_per_Minute was created by dividing Calories by Duration. This provides a normalized measure of calorie expenditure relative to workout length.

Pulse category

```
df["Pulse_Category"] = pd.cut(
    df["Pulse"],
    bins=[0, 99, 109, float("inf")],
    labels=["Low", "Moderate", "High"]
)
```

Pulse_Category groups Pulse into Low, Moderate and High categories using Pandas pd.cut().

Executed example

```
Duration Calories Calories_per_Minute
0 60 409.1 6.818333
1 60 479.0 7.983333
2 60 340.0 5.666667
3 45 282.4 6.275556
4 45 406.0 9.022222
```

```
Pulse Pulse_Category
0 110 High
1 117 High
2 103 Moderate
3 109 Moderate
4 117 High
5 102 Moderate
6 110 High
7 104 Moderate
8 109 Moderate
9 98 Low
```

5. Filtering Rows

The notebook demonstrates filtering using individual conditions and a combined condition.

Pulse above 110

```
high_pulse = df[df["Pulse"] > 110]
high_pulse
```

The filter returns the records with Pulse greater than 110, including the 117 and 130 Pulse observations.

Calories above 300

```
high_calories = df[df["Calories"] > 300]
high_calories
```

The filter identifies the higher-calorie records while retaining all qualifying observations.

Duration of at least 60 minutes

```
long_workouts = df[df["Duration"] >= 60]
long_workouts
```

The filter identifies longer workouts, including the documented 450-minute observation.

Combined filter

```
filtered_data = df[
    (df["Duration"] >= 60) &
    (df["Calories"] > 300)
]

filtered_data
```

The combined condition uses the logical AND operator (&) to identify records with both Duration $\geq$ 60 and Calories > 300.

6. Summary Statistics and Ranking

Mean values

```
df[["Duration", "Pulse", "Maxpulse", "Calories"]].mean()
```

Maximum values

```
df[["Duration", "Pulse", "Maxpulse", "Calories"]].max()
```

Minimum values

```
df[["Duration", "Pulse", "Maxpulse", "Calories"]].min()
```

Top 10 calorie records

```
df.sort_values("Calories", ascending=False).head(10)
```

7. Data Visualization

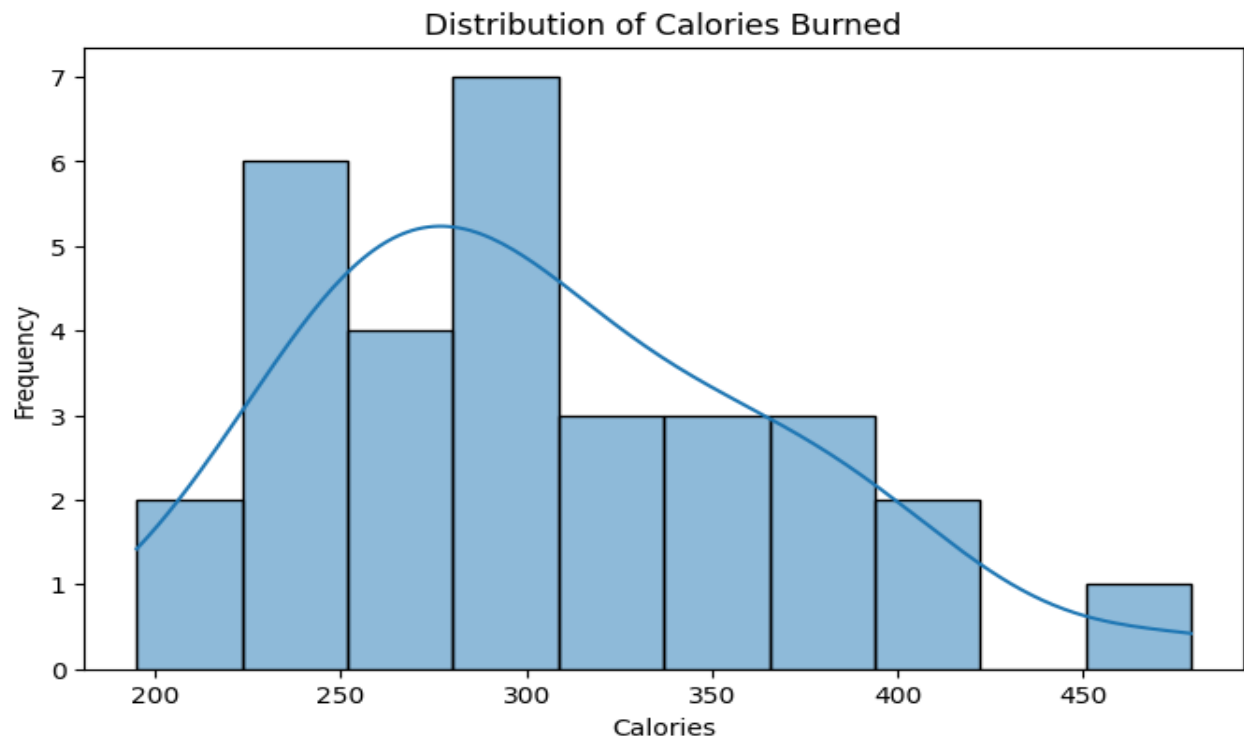
The executed notebook uses Matplotlib and Seaborn to create three basic visualizations.

Calories Burned vs Workout Duration



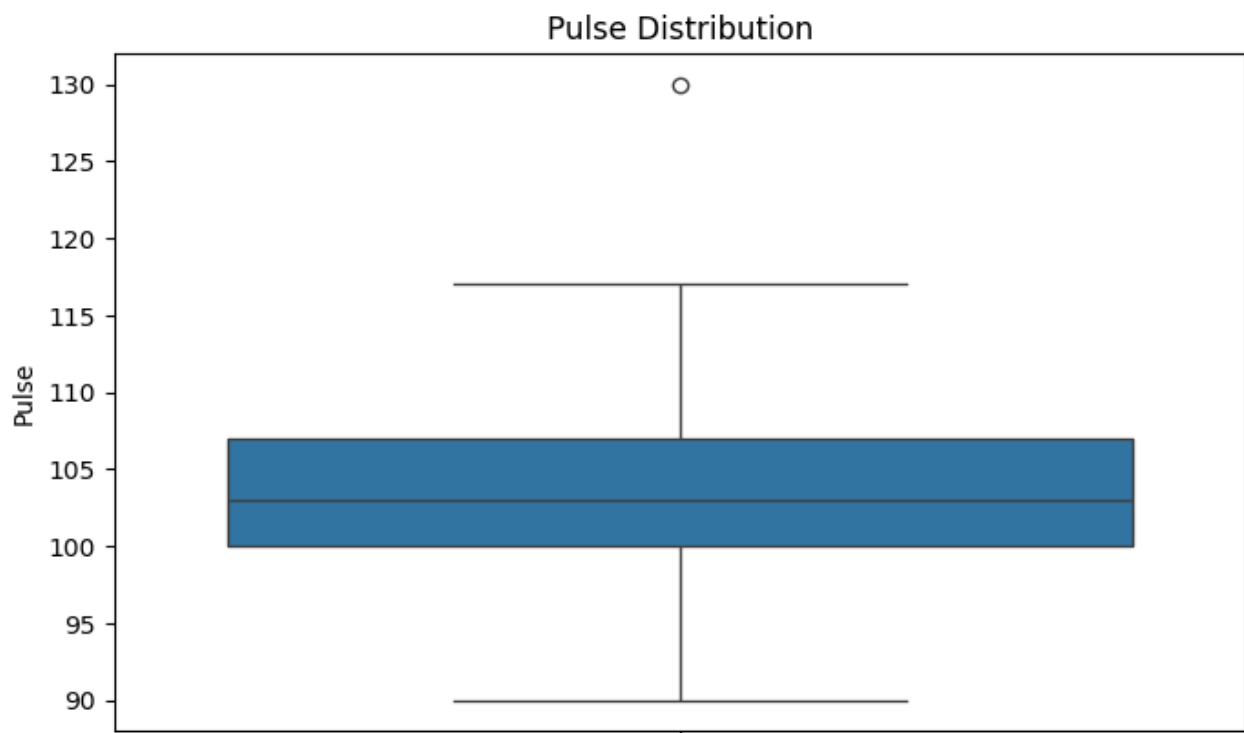
The scatter plot compares workout duration with calories burned and helps reveal the relationship between the two variables as well as unusual observations.

Distribution of Calories Burned



The histogram shows how calorie values are distributed across the cleaned dataset.

Pulse Distribution



The box plot summarizes the Pulse distribution and helps identify potential unusual values.

8. Final Validation and Conclusion

```
print("Rows:", df.shape[0])
print("Columns:", df.shape[1])
print("\nMissing values:")
print(df.isnull().sum())
print("\nDuplicate rows:", df.duplicated().sum())
```

Final validation	Result
Rows	31
Columns	7
Missing values	0 across all columns
Duplicate rows	0
Final columns	Duration, Date, Pulse, Maxpulse, Calories, Calories_per_Minute, Pulse_Category

Conclusion

The supplied dataset was successfully cleaned and analyzed using Python and Pandas. The Date field was standardized, the missing Date value was restored from the surrounding daily sequence, and the two missing Calories values were imputed using the median. One duplicate record was removed. The notebook then identified an unusually large Duration value, created two useful derived columns, filtered records using multiple conditions, calculated summary statistics and produced visualizations using Matplotlib and Seaborn.

The final validated dataset contains **31 rows and 7 columns**, with **zero missing values** and **zero duplicate rows**. The completed notebook contains the executable Python work supporting this report.

Appendix – Key Python Code

The following code blocks are reproduced from the uploaded Week 3 notebook. The notebook itself should be submitted alongside this PDF when the internship portal permits supporting files.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv("data.csv")
df.head()

df["Date"] = pd.to_datetime(
    df["Date"].astype(str).str.strip(""),
    format="mixed",
    errors="coerce"
)

missing_date_index = df["Date"].isna()

df.loc[missing_date_index, "Date"] = pd.Timestamp("2020-12-22")

calories_median = df["Calories"].median()
calories_median

df["Calories"] = df["Calories"].fillna(calories_median)

df = df.drop_duplicates()

df["Calories_per_Minute"] = df["Calories"] / df["Duration"]

df["Pulse_Category"] = pd.cut(
    df["Pulse"],
    bins=[0, 99, 109, float("inf")],
    labels=["Low", "Moderate", "High"]
)

high_pulse = df[df["Pulse"] > 110]
high_pulse

high_calories = df[df["Calories"] > 300]
high_calories

long_workouts = df[df["Duration"] >= 60]
long_workouts

filtered_data = df[
    (df["Duration"] >= 60) &
    (df["Calories"] > 300)
]

filtered_data

plt.figure(figsize=(8,5))

plt.scatter(df["Duration"], df["Calories"])

plt.xlabel("Duration (minutes)")
plt.ylabel("Calories")
plt.title("Calories Burned vs Workout Duration")

plt.show()

plt.figure(figsize=(8,5))

sns.histplot(df["Calories"], bins=10, kde=True)

plt.xlabel("Calories")
plt.ylabel("Frequency")
plt.title("Distribution of Calories Burned")

plt.show()

plt.figure(figsize=(8,5))

sns.boxplot(y=df["Pulse"])

plt.ylabel("Pulse")
plt.title("Pulse Distribution")

plt.show()

df.to_csv("cleaned_data.csv", index=False)
```